Inferencia verificable en modelos de Aprendizaje Automático mediante Pruebas de Conocimiento Cero

Álvaro Martínez Alfaro
alvaro.martinez39@alu.uclm.es
Criptografía, ESII, Universidad de Castilla-La Mancha
Albacete, España

29 de marzo de 2026

Resumen

1 Introducción

2 Fundamentos teóricos

En esta sección se presentan los conceptos teóricos necesarios para el desarrollo del trabajo, incluyendo una introducción a los modelos de aprendizaje automático, con un enfoque en las redes neuronales artificiales y, posteriormente, las pruebas de conocimiento cero, incidiendo en ambos casos en sus fundamentos matemáticos.

2.1 Aprendizaje Automático

El **aprendizaje automático** es una rama de la inteligencia artificial centrada en conseguir que los ordenadores simulen la capacidad humana de *aprender*, así como de buscar métodos que les permitan adquirir conocimientos y habilidades para mejorar su rendimiento ante determinadas tareas [1]. A diferencia del aprendizaje humano, que se construye a base de la experiencia adquirida a lo largo de grandes períodos de tiempo, los sistemas de aprendizaje automático, gracias a la potencia de cálculo de los ordenadores, pueden adquirir conocimientos utilizando grandes cantidades de datos en períodos de tiempo relativamente cortos; apenas unos minutos en modelos simples y con conjuntos de datos pequeños, como sería el caso de un *árbol de decisión*, u horas, días o incluso semanas en modelos complejos y con conjuntos de datos masivos, como es el caso de los *grandes modelos de lenguaje*.

2.1.1 Redes neuronales artificiales

Si hay un modelo de aprendizaje automático que ha adquirido notoriedad en las últimas décadas, ese es el de las **redes neuronales artificiales**, incluyendo todas sus variantes. Desde una perspectiva biológica simplificada, el cerebro humano se compone de una gran cantidad de *neuronas* interconectadas entre sí que se comunican mediante *impulsos eléctricos* que reciben a través de sus *dendritas* y transmiten a través de sus *axones* [2]. De manera similar, las redes neuronales artificiales se componen de unidades denominadas *neuronas artificiales*, conectadas entre sí mediante *enlaces* que transmiten señales. Concretamente, estas señales se representan mediante *números reales*, donde cada uno de ellos simboliza la fuerza de la conexión, también conocida como *peso*. Una red neuronal suele contener desde algunas decenas hasta miles o incluso millones de neuronas artificiales, organizadas en diferentes *capas*. Las neuronas de cada capa se encuentran conectadas a las de la capa siguiente. Un ejemplo típico de red neuronal se muestra en la Figura 1, la cual está compuesta por una capa de entrada con N_i neuronas, una capa de salida con N_o neuronas y varias capas ocultas con N_h neuronas. Cuando una red neuronal contiene una o más capas

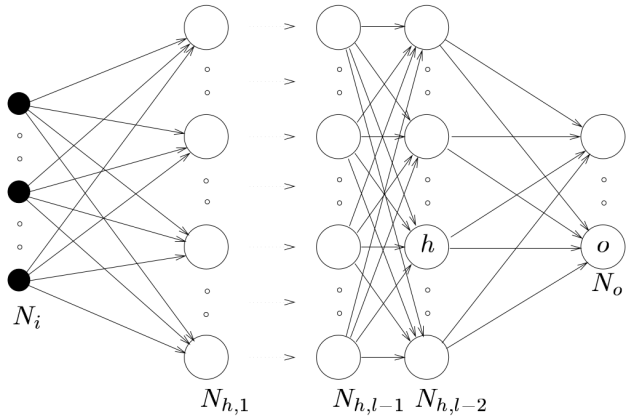


Figura 1: Ejemplo de red neuronal artificial [3].

ocultas, se denomina *red neuronal profunda*, siendo este el caso mostrado y, en general, el más común en la actualidad.

En cada neurona, se realiza una suma ponderada de las señales recibidas a través de sus enlaces. Así, en una cierta neurona k, la entrada total que recibe en un instante de tiempo t viene dada por

$$s_k(t) = \sum_j w_{jk}(t) y_j(t) + \theta_k(t) \tag{1}$$

donde $w_{jk}(t)$ representa el peso del enlace que conecta la neurona j con la neurona k, siendo j, normalmente, una neurona de la capa anterior [3]. Cuando el peso es positivo, se dice que el enlace es *excitatorio*, mientras que, si es negativo, se dice que es *inhibitorio*. Luego, $y_j(t)$ representa la señal de salida de la neurona j y, finalmente, $\theta_k(t)$ representa el *sesgo* de la neurona k, un valor constante que permite ajustar su umbral de activación.

Tras ello, la neurona k aplica una *función de activación* a su entrada total, $s_k(t)$, para generar su salida, tal que

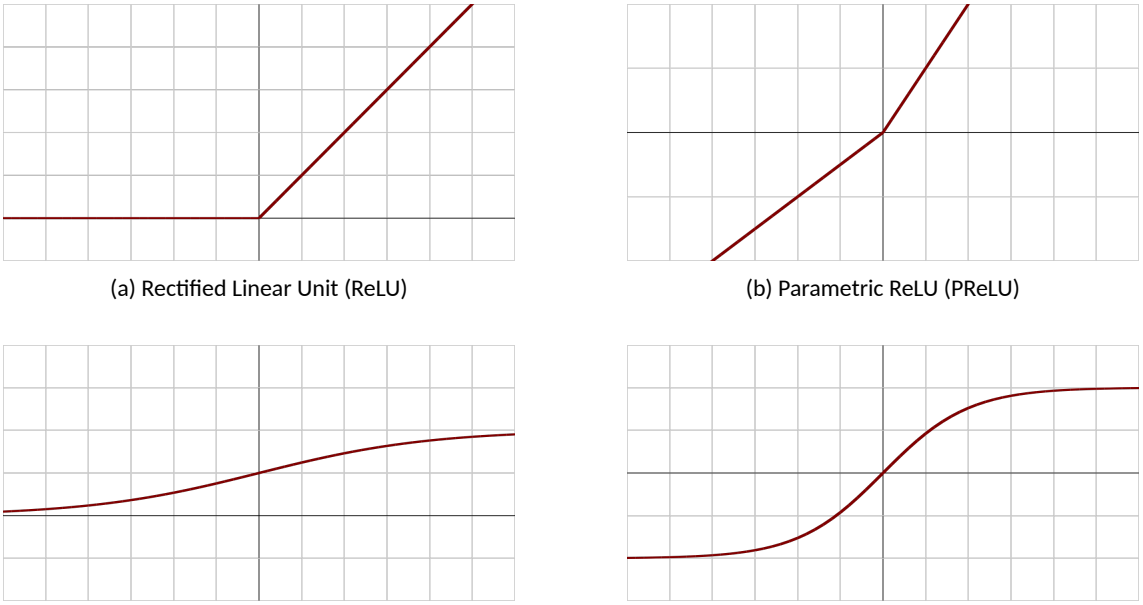
$$y_k(t+1) = \mathcal{F}(s_k(t)) \tag{2}$$

En general, $\mathcal{F}$ suele ser una función no lineal debido a que esto es lo que permite a las redes neuronales aprender patrones complejos en los datos. Algunas de las funciones de activación más utilizadas se muestran en la Figura 2. Sin embargo, no se profundizará en ellas debido a que no resulta especialmente relevante para el desarrollo del trabajo. Hay extensa literatura sobre el tema, por lo que una simple búsqueda de artículos o libros sobre redes neuronales artificiales o aprendizaje profundo proporcionará una gran cantidad de información al respecto [4, 5]. A pesar de ello, un aspecto importante a destacar es que es condición necesaria que la función de activación sea diferenciable o, al menos, diferenciable en casi todo punto, ya que esto será necesario en el proceso de entrenamiento, basado en la *propagación hacia atrás* del error y el uso de algoritmos de optimización como el *descenso de gradiente* [3].

2.1.2 Entrenamiento en una red neuronal artificial

Llegados a este punto, se habrá observado que no se ha hecho mención alguna a cómo se determinan los pesos de los enlaces. Esto se debe a que, en general, no se establecen manualmente, más allá de una inicialización aleatoria, sino que se aprenden mediante el proceso de *entrenamiento*. Dado un conjunto de datos de entrenamiento, $\mathcal{X}$, este consiste en cuatro simples pasos [3]:

1. Inicializar los pesos de la red neuronal de forma aleatoria.
2. Seleccionar un ejemplo de entrenamiento $x \in \mathcal{X}$ e introducirlo por la capa de entrada, de forma que cada neurona genere una señal de salida que transmitirá a las neuronas de la capa siguiente, y así sucesivamente, hasta llegar a la capa de salida, donde se generará una salida y.
3. Si y es diferente a la salida deseada o esperada, $d(x)$, modificar los pesos de la red neuronal de acuerdo al error.



(a) Rectified Linear Unit (ReLU)

(b) Parametric ReLU (PReLU)

(c) Función logística

(d) Función tangente hiperbólica

Figura 2: Funciones de activación en redes neuronales.

4. Volver al paso 2 hasta alcanzar un cierto criterio de parada, como puede ser un número máximo de iteraciones o un umbral de error mínimo.

Intuitivamente, es fácil ver que el entrenamiento no es más que un proceso iterativo de prueba y error, donde se ajustan los pesos de la red neuronal para que su salida converja a la salida deseada para cada ejemplo de entrenamiento. Matemáticamente, no es un proceso trivial. A continuación, se presenta una explicación detallada de cómo se consigue actualizar los pesos de la red; sin embargo, al final de la sección se mostrará un resumen de todo el proceso, haciendo mención a las dos ecuaciones que lo rigen, pudiendo ser de utilidad para aquellos lectores que no deseen profundizar en los detalles matemáticos.

Se asume una red neuronal formada por una única capa y una única neurona de salida con función de activación lineal. En este caso, la salida viene dada por

$$y = \sum_j w_j x_j + \theta \tag{3}$$

teniendo así la red capacidad de aprender relaciones lineales entre la entrada y la salida. Contando con un conjunto de entrenamiento formado por valores x^p y salidas deseadas d^p, el objetivo será encontrar el hiperplano que mejor se ajuste a los datos, es decir, encontrar los pesos w_j que minimicen el error entre la salida de la red neuronal y^p y la salida deseada d^p para cada ejemplo de entrenamiento p. Para ello, se define una función de error, E, tal que

$$E = \frac{1}{2} \sum_p (d^p - y^p)^2 \tag{4}$$

esto es, el error cuadrático medio entre la salida de la red neuronal y la salida deseada. Una vez calculada la función de error, se actualizan los pesos de la red neuronal mediante el algoritmo de descenso de gradiente.

El objetivo del descenso de gradiente es minimizar la función de error E modificando los pesos proporcionalmente a la dirección del gradiente negativo de E, esto es,

$$\Delta_p w_j = -\eta \frac{\partial E^p}{\partial w_j} = -\eta \frac{\partial E^p}{\partial y^p} \frac{\partial y^p}{\partial w_j} = -\eta \frac{\partial E^p}{\partial y^p} x_j = \eta (d^p - y^p) x_j = \eta \delta^p x_j \tag{5}$$

siendo η una constante de proporcionalidad, también conocida como *tasa de aprendizaje*, que controla el tamaño de los pasos que se dan en la dirección del gradiente negativo y denotando δ^p a la diferencia entre la salida deseada y la salida obtenida.

Es fácil generalizar a redes neuronales con múltiples capas y neuronas de salida, con funciones de activación no lineales. La activación de cada neurona es una función de la entrada total y, como se ha mencionado, diferenciable, tal que

$$y_k^p = \mathcal{F}(s_k^p) = \mathcal{F}(\sum_j w_{jk} y_j^p + \theta_k) \tag{6}$$

siendo w_{jk} el peso del enlace entre la neurona j de la capa anterior y la neurona k y y_j^p la salida de la neurona j. Con ello, a partir de la función de error,

$$E^p = \frac{1}{2} \sum_{o=1}^{N_o} (d_o^p - y_o^p)^2 \tag{7}$$

donde d_o^p y y_o^p representan la salida deseada y la salida obtenida en la neurona de salida o, el descenso de gradiente se generaliza a

$$\Delta_p w_{jk} = -\eta \frac{\partial E^p}{\partial w_{jk}} = -\eta \frac{\partial E^p}{\partial s_k^p} \frac{\partial s_k^p}{\partial w_{jk}} = -\eta \frac{\partial E^p}{\partial s_k^p} y_j^p = \eta \delta_k^p y_j^p \tag{8}$$

Siendo $\delta_k^p = -\frac{\partial E^p}{\partial s_k^p}$, se obtiene una regla de actualización de los pesos equivalente a la de la Ecuación 5. Ahora, cabe preguntarse cómo se calcula δ_k^p para cada neurona k. Aquí es donde entra en juego el algoritmo de *propagación hacia atrás*, el cual se basa en calcular el error en las neuronas de salida y, luego, ir propagándolo hacia atrás, a través de las capas ocultas, hasta llegar a la capa de entrada.

Aplicamos la regla de la cadena para calcular δ_k^p como el producto de dos términos, uno que representa el cambio en la función de error con respecto a la salida de la neurona k y otro que representa el cambio en la salida de la misma neurona con respecto a su entrada total, esto es,

$$\delta_k^p = -\frac{\partial E^p}{\partial s_k^p} = -\frac{\partial E^p}{\partial y_k^p} \frac{\partial y_k^p}{\partial s_k^p} \tag{9}$$

Por un lado, como $y_k^p = \mathcal{F}(s_k^p)$, se tiene que

$$\frac{\partial y_k^p}{\partial s_k^p} = \mathcal{F}'(s_k^p) \tag{10}$$

Por otro lado, para el término $\frac{\partial E^p}{\partial y_k^p}$, si k es una neurona de salida, $k = o$, se obtiene

$$\frac{\partial E^p}{\partial y_k^p} = -(d_o^p - y_o^p) \tag{11}$$

De esta forma, la regla de actualización de los pesos para cada neurona de salida o viene dada por

$$\Delta_p w_{jo} = \eta \delta_o^p y_j^p = \eta \mathcal{F}'(s_o^p)(d_o^p - y_o^p) y_j^p \tag{12}$$

Ahora, si k es una neurona oculta, no es posible saber cuánto contribuye k al error fácilmente. Si tenemos en cuenta que el error puede ser calculado como función de las salidas de las neuronas ocultas a la capa de salida, $E^p = E^p(s_1^p, s_2^p, \ldots, s_j^p, \ldots)$, mediante la regla de la cadena, se obtiene

$$\frac{\partial E^p}{\partial y_h^p} = \sum_{o=1}^{N_o} \frac{\partial E^p}{\partial s_o^p} \frac{\partial s_o^p}{\partial y_h^p} = \sum_{o=1}^{N_o} \frac{\partial E^p}{\partial s_o^p} \frac{\partial}{\partial y_h^p} \sum_{j=1}^{N_h} w_{ko} y_j^p = \sum_{o=1}^{N_o} \frac{\partial E^p}{\partial s_o^p} w_{ho} = -\sum_{o=1}^{N_o} \delta_o^p w_{ho} \tag{13}$$

Con ello, se obtiene

$$\delta_h^p = \mathcal{F}'(s_h^p) \sum_{o=1}^{N_o} \delta_o^p w_{ho} \tag{14}$$

y, por tanto, la regla de actualización de los pesos para cada neurona oculta h viene dada por

$$\Delta_p w_{jh} = \eta \delta_h^p y_j^p = \eta \mathcal{F}'(s_h^p)(\sum_{o=1}^{N_o} \delta_o^p w_{ho}) y_j^p \tag{15}$$

Como ya se ha comentado, el proceso de entrenamiento no es trivial, aunque es posible ver que este permite ajustar los pesos de la red neuronal de forma recursiva. Se toma un ejemplo de entrenamiento, se introduce por la capa de entrada y se propaga hacia adelante a través de las capas ocultas hasta llegar a la capa de salida. En ella, se calcula el error de cada neurona de salida o acorde al producto de la derivada de su función de activación por la diferencia entre la salida deseada y la salida obtenida, obteniendo así el valor δ_o^p. Con ello, los pesos se actualizan de acuerdo a la Ecuación 12. Luego, para cada neurona oculta h, se calcula su error como el producto de la derivada de su función de activación por la suma del producto de los errores de las neuronas de salida por los pesos que las conectan, obteniendo así δ_h^p. Con ello, los pesos se actualizan de acuerdo a la Ecuación 15. Este proceso se repite iterativamente para cada ejemplo de entrenamiento, ajustando los pesos de la red neuronal hasta que se alcance algún criterio de parada.

Este proceso presenta algunas limitaciones, como el desvanecimiento del gradiente cuando la red contiene muchas capas. Sin embargo, tampoco se profundizará en ellas debido a que no resulta relevante para el desarrollo del trabajo. De nuevo, hay extensa literatura sobre el tema; para más información, un artículo interesante a consultar es el de *Pascanu et al.* relativo a este problema y el de la explosión del gradiente [6].

2.2 Pruebas de conocimiento cero

Referencias

[1] H. Wang, C. Ma, and L. Zhou, "A brief review of machine learning and its application," in *2009 International Conference on Information Engineering and Computer Science*, pp. 1–4, 2009.

[2] M. Islam, G. Chen, and S. Jin, "An overview of neural network," *American Journal of Neural Networks and Applications*, vol. 5, p. 05, 06 2019.

[3] B. Kröse and P. van der Smagt, *An Introduction to Neural Networks*. The University of Amsterdam, eighth ed., November 1996.

[4] V. Kunc and J. Kléma, "Three decades of activations: A comprehensive survey of 400 activation functions for neural networks," 2024.

[5] S. R. Dubey, S. K. Singh, and B. B. Chaudhuri, "Activation functions in deep learning: A comprehensive survey and benchmark," 2022.

[6] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," 2013.